

Peer-Review-Write-Up

May 4, 2026

0.1 Code Review and Efficiency (10 points): Are there any parts of the code that seem inefficient or redundant? How could these sections be optimized for better performance or clarity?

I do not think there were many inefficiencies, but I found some parts to be slightly convoluted at times. The biggest trouble I had was with parts of the data build. Specifically, in “hcris-v2010”, we create the year variables and load in the data for 2010-2019. In “hcris-combine”, however, we create the fiscal year variable “fyear” and drop years. This was fine, and the rationale completely makes sense, as we discussed in class. However, this created a major issue when I created the data in the end: instead of having a CSV for the years 2010-2018, it gave me one for the years 2011-2019.

This leads me to something I realized really helps with optimizing the code for clarity that I did not do enough throughout the semester: comments. When working with someone else’s GitHub repository more directly, the use of comments for important chunks of code can make a monumental difference in terms of understanding someone’s thought process and rationale for doing things. Returning to the fiscal year example, there is a comment on the chunk of code that says this: “## compute uncomp_care from S-10 components”. This is helpful on some level, but could, for instance, have included a brief portion on why we were creating the specific variables.

0.2 Comparison of Results (10 points): How well do their results compare with your own? Are there any areas where the two analyses are particularly different?

Our results were similar, and there was minimal variation in the numbers we obtained. In question 1, our mean uncompensated care costs were mostly comparable; the largest discrepancy was around \$5M. In question 2, there was minimal fluctuation in the movement of our graphs post-expansion implementation, but otherwise, mostly similar. In question 3, there was a larger difference between our scales for non-expansion states (in the 50s for mine vs. in the 40s for my peer). However, the expansion states had nearly identical values. Furthermore, the trajectories changed in the same way for expansion and non-expansion states for both my peer and me. In question 5, my peer had a post x expansion effect of around -23, while I had one of approximately -26. In question 6, I had a value of -32, and my peer had one of almost -26. This is probably the most significant discrepancy between my work and my peer’s work. Question 7 was relatively similar for both of us, and so were 8 and 9.

0.3 Replicability and Documentation (10 points): Are you able to clone your peer's repository and replicate their results?

For the most part. The biggest issue was more so on my end: my kable extra was not working due to an issue with my xfun package, specifically, and as a result, rather than being able to make tables using the model summary, I had to convert my work from R to Python to have a similar presentation of my work as what we have been using in the classroom. Elsewhere, as mentioned earlier, I had trouble with the fiscal year concept, which yielded data on a forward-shifted scale, basically not providing me with 2010 data when I tried to clone it.

0.4 Diagnosing Replication (10 points): If you were unable to replicate their results using their code, what do you think is the central problem? If you were able to replicate, are there things that could have been changed (e.g., code structure, documentation, commenting, etc) that would have made it easier?

I found that the main question-by-question analysis in the homework was well conveyed. One thing I appreciated was that the code and cleaning each were in their own cell, and the actual table/graph creation got its own cell. This made the workflow feel smooth and organized. Once I got through understanding the trouble I had in data-build, running the actual answers to the question was much easier (barring my trouble with kable extra).

Again, my issues with replication came back down to the year conversions and the general complexity of the data-build. As suggested earlier, I think the only fix would be using more specific comments over these particular notes in the data-build, because otherwise, I found that the rationale for every decision made was straightforward. For instance, another thing that gave me trouble was the creation of the Medicaid expansion.txt file in data/inputs. For me, I used the KFF CSV from online, so I was somewhat confused when I saw this document in the inputs, before remembering that we discussed this in class.

0.5 Final Thoughts (10 points): How might you update your own code based on your review of your peer's work?

Reflecting now on my own code compared to my peer's, there are intricacies that I realize are done that make the material be conveyed smoothly that I could incorporate into my own code in the future:

Firstly, I noticed that for most of the questions, such as questions 8 or 9, where no follow-up commentary is requested, my peer usually adds a small tidbit with a kind of analysis on what is important in the results. So much of this class was about how to try to apply these coding principles in an actual workplace, so with that in mind, I think giving analysis rather than just code and tables is a great practice for the future, which I will try to bring with me.

Secondly, I think my peer's workflow or pipeline of the analysis flows nicely. Every single variable is named clearly and is consistently maintained throughout the whole

file. Everything seems to be well organized from top to bottom, and I think at times my workspaces can get cluttered. Part of this comes from experience in a coding environment, in my opinion, but overall, it is still an important skill that I plan to work towards as I spend more time coding. Overall, the project flows cleanly and comes off as an actual project, and not just scattered answers to questions. I think my work can come off that way at times, and this is something I will keep in mind moving forward.

[]: